

SIMATS ENGINEERING
Department of Computer Science & Engineering
ASSESSMENT TOOL 1- ANALYTICAL PROBLEM SOLVING

Course Code:	CSA1202	Course Title:	Computer Architecture
CO Assessed:	CO2	Assessment:	ASSESSMENT TOOL1- ANALYTICAL PROBLEM SOLVING
Weightage:	45%	Total Marks:	25
CO2 Statement:	Apply integer and floating-point arithmetic algorithms including Booth's multiplication, restoring/non-restoring division, and IEEE 754 standard operations to solve fixed-point and floating-point computational problems. (BL3)		
Student Name:	Athisaya U	Reg.No.:	192571001
Department:	BTech Computer science and biosciences	Date:	27.6.2026

ANNEXURE A

1. A processor performs arithmetic operations on signed 8-bit integers using 2's complement representation. Consider two numbers: +45 and -23. Analyze in detail how the system performs both addition and subtraction, including binary conversion, alignment of sign bits, and intermediate steps. Determine whether overflow occurs, and explain how the processor detects overflow and interprets the final result.

Step 1 - Convert the decimal values:

+45 = 00101101

+23 = 00010111

Step 2 - Find the 2's complement of +23:

+23 = 00010111

1's comp. = 11101000

+ 1 = 11101001 = -23

Step 3 - Add +45 and -23:

00101101 (+45)

+ 11101001 (-23)

1 00010110 -> discard carry-out

Result = 00010110 = +22

Step 4 - Check addition overflow:

$$\text{Sign}(+45) = 0; \text{Sign}(-23) = 1$$

Different signs $\rightarrow$ no signed overflow

$$V = 0$$

Step 5 - Subtract -23 from +45:

$$+45 - (-23) = +45 + (+23)$$

$$00101101 \quad (+45)$$

$$+ 00010111 \quad (+23)$$

$$01000100 = +68$$

Step 6 - Final results:

$$+45 + (-23) = +22; V = 0$$

$$+45 - (-23) = +68; V = 0$$

Both results are within -128 to +127.

2. A high-performance computing system replaces a ripple carry adder with a 4-bit carry lookahead adder (CLA) to reduce computation delay. Given two binary inputs, analyze how generate and propagate signals are formed and how carries are computed in advance. Compare the time delay of CLA with ripple carry addition and justify, with reasoning, why CLA improves speed in modern processors.

Step 1 - Write the inputs and formulas:

$$A = 1011; B = 0110; C_0 = 0$$

$$P_i = A_i \text{ XOR } B_i$$

$$G_i = A_i \text{ AND } B_i$$

Step 2 - Calculate P_i and G_i :

Bit i	A_i	B_i	P_i	G_i
0	1	0	1	0
1	1	1	0	1
2	0	1	1	0
3	1	0	1	0

Step 3 - Expand the carry equations:

$$C_1 = G_0 + P_0 C_0$$

$$C_2 = G_1 + P_1 G_0 + P_1 P_0 C_0$$

$$C_3 = G_2 + P_2 G_1 + P_2 P_1 G_0 + P_2 P_1 P_0 C_0$$

$$C_4 = G_3 + P_3 G_2 + P_3 P_2 G_1 + P_3 P_2 P_1 G_0 + P_3 P_2 P_1 P_0 C_0$$

Step 4 - Substitute the values:

$C1 = 0; C2 = 1; C3 = 1; C4 = 1$

Step 5 - Find the sum:

$S_i = P_i \text{ XOR } C_i$

$S_3S_2S_1S_0 = 0001; C_4 = 1$

Final result = 1 0001 = 17 = 11 + 6

Step 6 - Compare the delays:

Feature	Ripple Carry Adder	Carry Lookahead Adder
Carry formation	Sequential	Parallel
Delay growth	$O(n)$	Toward $O(\log n)$
Hardware	Fewer gates	More gates and wiring
Result	Slower critical path	Faster ALU operation

3. In a signed multiplication operation, a processor uses Booth's algorithm to multiply two numbers, -6 and +5. Analyze the algorithm step-by-step, including the role of the accumulator, multiplier, and Booth bit, along with arithmetic shifts. Explain how the algorithm efficiently handles signed numbers and reduces the number of addition/subtraction operations compared to traditional multiplication.

Step 1 - Initialize the registers:

$M = -6 = 1010; -M = +6 = 0110$

$Q = +5 = 0101$

$A = 0000; Q-1 = 0; n = 4$

Step 2 - Apply Booth's rule:

$Q_0Q_{-1} = 01 \rightarrow A = A + M$

$Q_0Q_{-1} = 10 \rightarrow A = A - M$

$Q_0Q_{-1} = 00 \text{ or } 11 \rightarrow \text{no operation}$

After every operation $\rightarrow$ arithmetic right shift [A, Q, Q-1]

Step 3 - Perform the four iterations:

Cycle	A operation	A after ASR	Q, Q-1 after ASR
1: 10	$A - M = 0110$	0011	0010, 1
2: 01	$A + M = 1101$	1110	1001, 0
3: 10	$A - M = 0100$	0010	0100, 1
4: 01	$A + M = 1100$	1110	0010, 0

Step 4 - Combine A and Q:

$[A,Q] = 1110\ 0010$

$11100010 \rightarrow 2\text{'s-complement magnitude} = 00011110 = 30$

Final product = -30

Check: $(-6) \times (+5) = -30$

Step 5 - Efficiency note:

Booth directly handles signed 2's-complement operands.

A run of k consecutive 1s is replaced by one subtraction and one addition.

For $Q = 0101$, the benefit is limited because the 1s are not consecutive.

4. A processor is required to perform division of two binary numbers (e.g., $13 \div 3$) using both restoring and non-restoring division algorithms. Analyze both methods step-by-step, showing intermediate remainders and quotient formation. Compare the two approaches in terms of number of steps, complexity, and efficiency, and explain why non-restoring division is often preferred in modern systems.

Step 1 - Initialize the registers:

Dividend $Q = 13 = 1101$

Divisor $M = 3 = 00011$

Partial remainder $A = 00000$; $n = 4$

Step 2 - Apply restoring division:

Shift $[A,Q] \rightarrow A=A-M$

$A < 0 \rightarrow Q_0 = 0$ and restore $A = A + M$

$A \geq 0 \rightarrow Q_0 = 1$

Shifted A,Q	Trial A-M	Action	Final A,Q
00001,1010	11110 (-2)	$Q_0 = 0$; restore	00001,1010
00011,0100	00000	$Q_0 = 1$	00000,0101
00000,1010	11101 (-3)	$Q_0 = 0$; restore	00000,1010
00001,0100	11110 (-2)	$Q_0 = 0$; restore	00001,0100

Step 3 - Read the restoring result:

Quotient $Q = 0100 = 4$

Remainder $A = 00001 = 1$

Check: $13 = (3 \times 4) + 1$

Step 4 - Apply non-restoring division:

Previous $A \geq 0 \rightarrow$ shift and subtract M

Previous $A < 0 \rightarrow$ shift and add M

Set Q_0 from the new sign; restore only once at the end if $A < 0$

Step 3 - Write the addition operands:

$$-13.25 = -1.10101 \times 2^3$$

$$+5.5 = +1.011 \times 2^2$$

Step 4 - Align the exponents:

$$+1.011 \times 2^2 = +0.10110 \times 2^3$$

Aligned operands: -1.10101×2^3 and $+0.10110 \times 2^3$

Step 5 - Subtract and normalize:

$$1.10101 \times 2^3$$

$$- 0.10110 \times 2^3$$

$$0.11111 \times 2^3 = 1.1111 \times 2^2$$

$$\text{Final value} = -1.1111 \times 2^2 = -111.11(\text{base } 2) = -7.75$$

Step 6 - Rounding and precision check:

Guard, round and sticky bits are used when shifted bits are lost.

Default mode: round to nearest, ties to even.

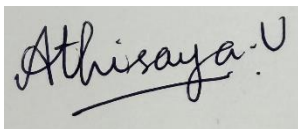
13.25 and -7.75 terminate in binary $\rightarrow$ no rounding error here.

Single precision: 24 significant bits; double precision: 53 bits.

Code of Conduct:

I Athisaya U (192571001) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:



MARKS ALLOCATION

	Problem Understanding (20%)	Method Selection (20%)	Solution Process (25%)	Accuracy of Calculation (20%)	Interpretation of Results (15%)
Q1					
Q2					
Q3					
Q4					
Q5					

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	20%	Clearly interprets problem, identifies all parameters and requirements accurately	Understands problem with minor gaps	Partial understanding; misses key elements	Misinterprets or unable to understand problem
Method Selection	20%	Selects most appropriate method/formula with strong justification	Selects correct method with minor errors	Inappropriate or partially correct method	Unable to select method
Solution Process	25%	Logical, systematic steps with correct approach throughout	Mostly correct steps with minor mistakes	Disorganized steps; multiple errors	No clear process
Accuracy of Calculation	20%	All calculations accurate; correct final answer	Minor calculation errors	Frequent errors; incorrect final answer	No valid calculations
Interpretation of Results	15%	Clearly interprets results with units and engineering relevance	Basic interpretation with minor omissions	Limited or unclear interpretation	No interpretation